

Attention

Advanced NLP M26

Aparajitha + Manish Shrivastava



INTERNATIONAL INSTITUTE OF
INFORMATION TECHNOLOGY

HYDERABAD

August 20, 2026

Transformer

Where are we?

Attention

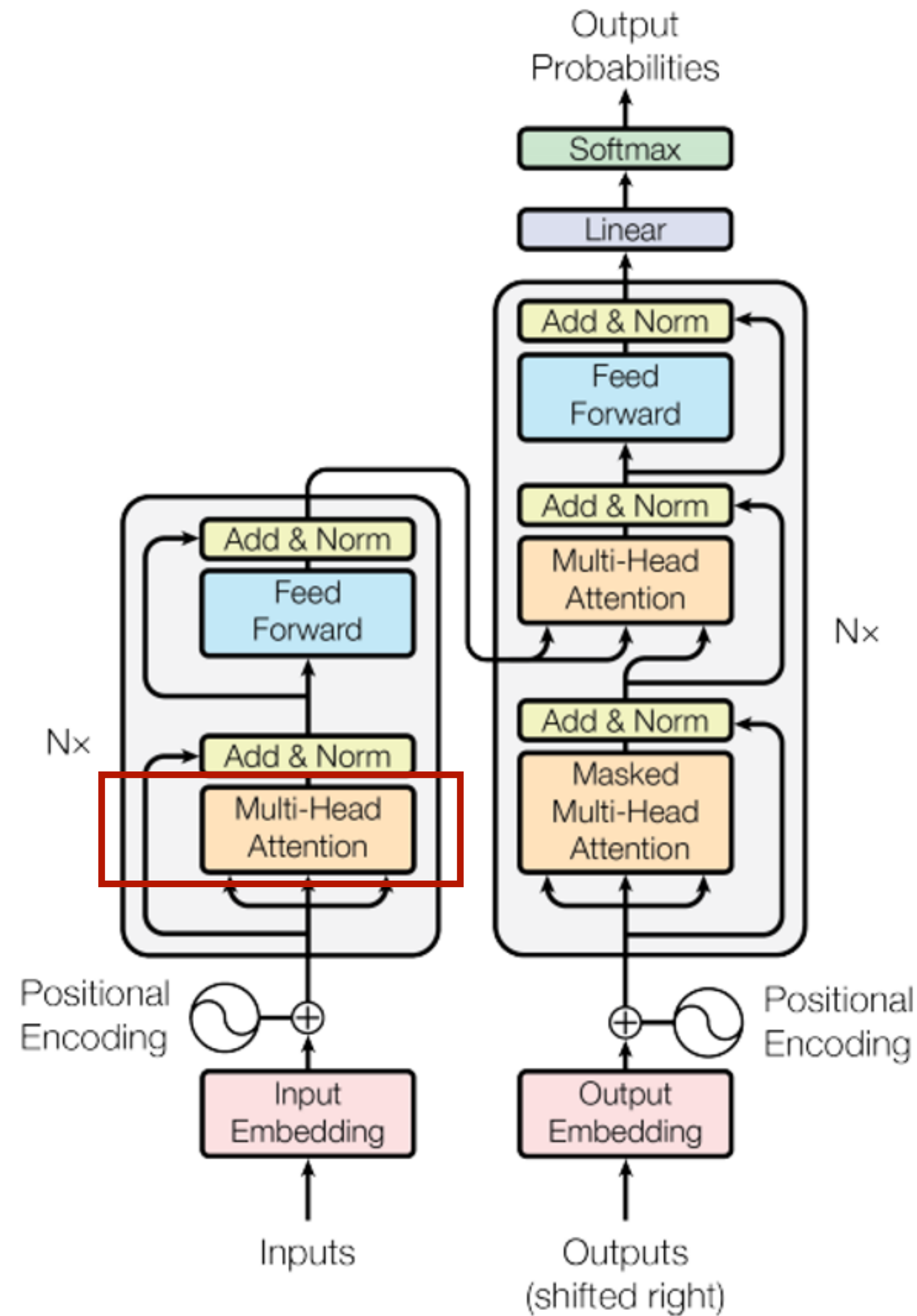


Figure 1: The Transformer - model architecture.

Transformer

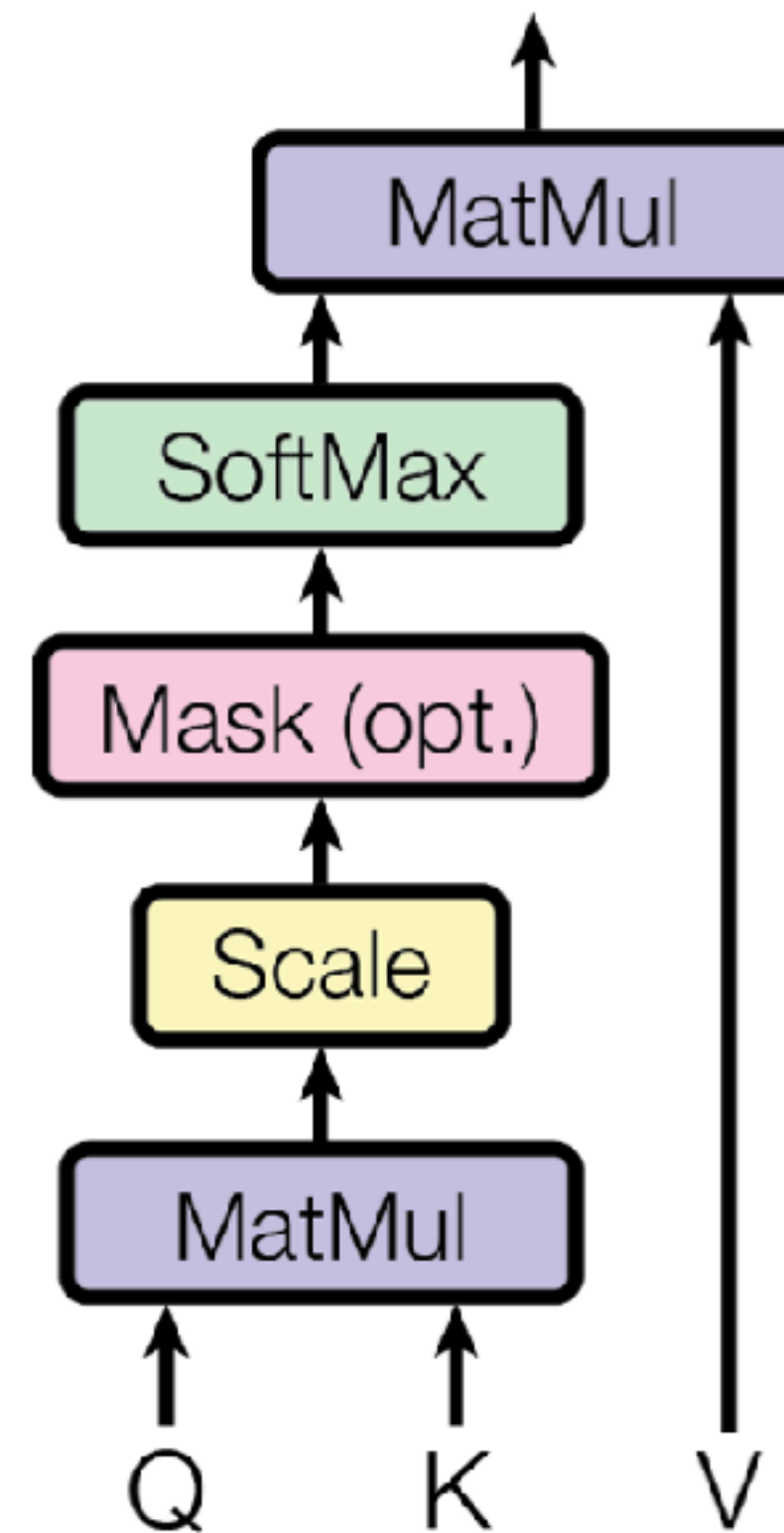
Parameter Counts ~1B Model

V	Vocab Size	32000
d	Embedding Dim	2048
n	Context Len	2048
dq	Q K V Dim	128
h, l	Heads, Layers	16

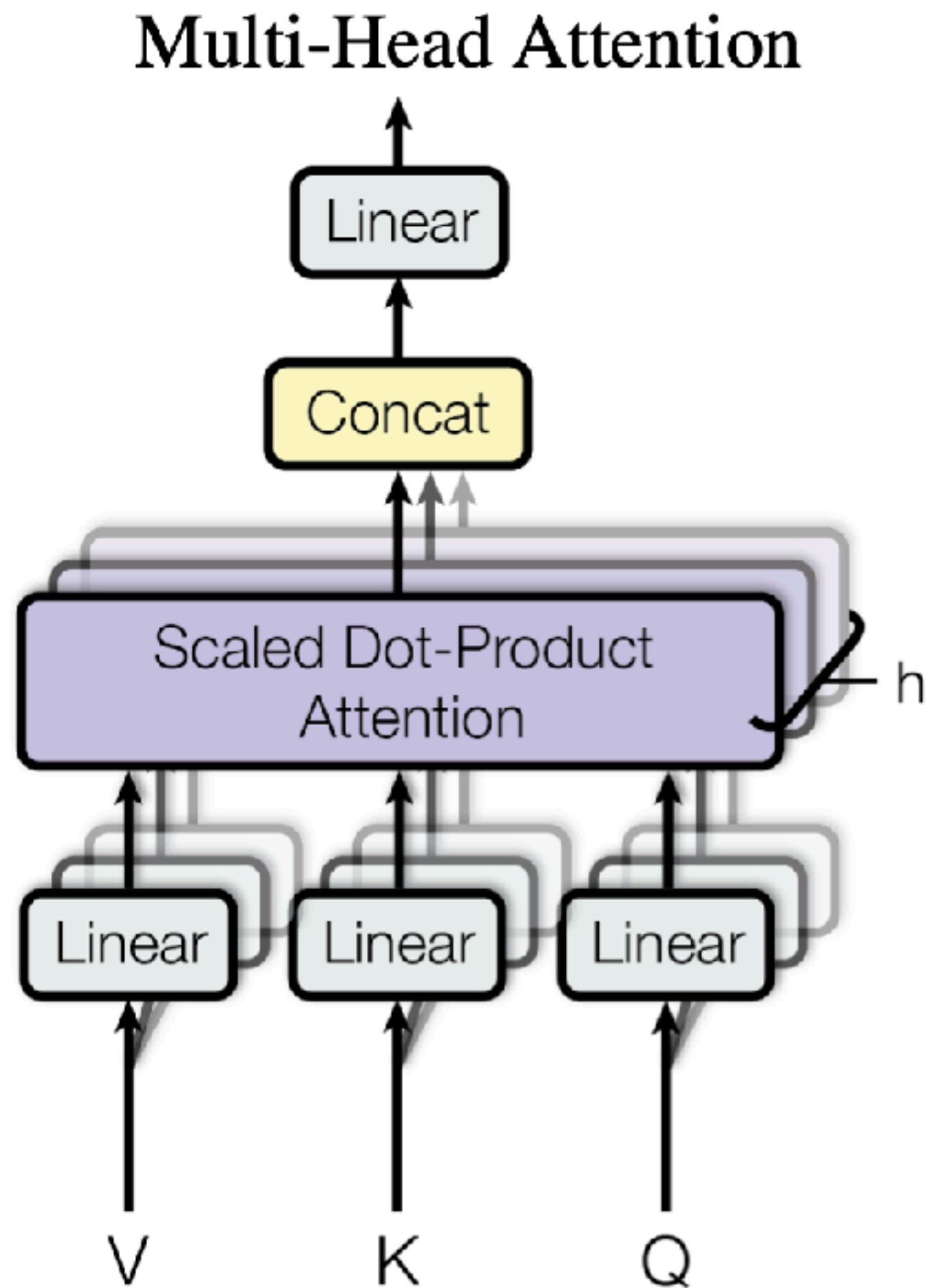
Component	Calculation	~Param	~%
Token Embedding	$V * d$	65M	6.95
Positional Embedding	$n * d$	4.2M	0.44
Attention	$4 * d_q * d * h * l$	268M	28.63
FFNN	$2 * d * 4d * l$	536M	57.37
Decoding	$d * V$	65M	6.95
Bias + Layer Norm		~0.3M	0.03
Total		935.8M	

Self Attention

Scaled Dot-Product Attention



Multi Head Attention



Multi Head Attention

Gains

- Parallelization
- Long distance dependency



Multi Head Attention

Drawbacks

- Quadratic computation in Self Attention
 - High computation and memory costs $O(n^2)$
- Redundant Heads
- Slow inference
 - Extremely large KV cache



KV Computation

$$QK^{\top} = \begin{bmatrix} \mathbf{q}_0 \cdot \mathbf{k}_0 & \mathbf{q}_0 \cdot \mathbf{k}_1 & \mathbf{q}_0 \cdot \mathbf{k}_2 & \mathbf{q}_0 \cdot \mathbf{k}_3 \\ \mathbf{q}_1 \cdot \mathbf{k}_0 & \mathbf{q}_1 \cdot \mathbf{k}_1 & \mathbf{q}_1 \cdot \mathbf{k}_2 & \mathbf{q}_1 \cdot \mathbf{k}_3 \\ \mathbf{q}_2 \cdot \mathbf{k}_0 & \mathbf{q}_2 \cdot \mathbf{k}_1 & \mathbf{q}_2 \cdot \mathbf{k}_2 & \mathbf{q}_2 \cdot \mathbf{k}_3 \\ \mathbf{q}_3 \cdot \mathbf{k}_0 & \mathbf{q}_3 \cdot \mathbf{k}_1 & \mathbf{q}_3 \cdot \mathbf{k}_2 & \mathbf{q}_3 \cdot \mathbf{k}_3 \end{bmatrix}$$

$$V = \begin{bmatrix} \mathbf{v}_0 \\ \mathbf{v}_1 \\ \mathbf{v}_2 \\ \mathbf{v}_3 \end{bmatrix}$$



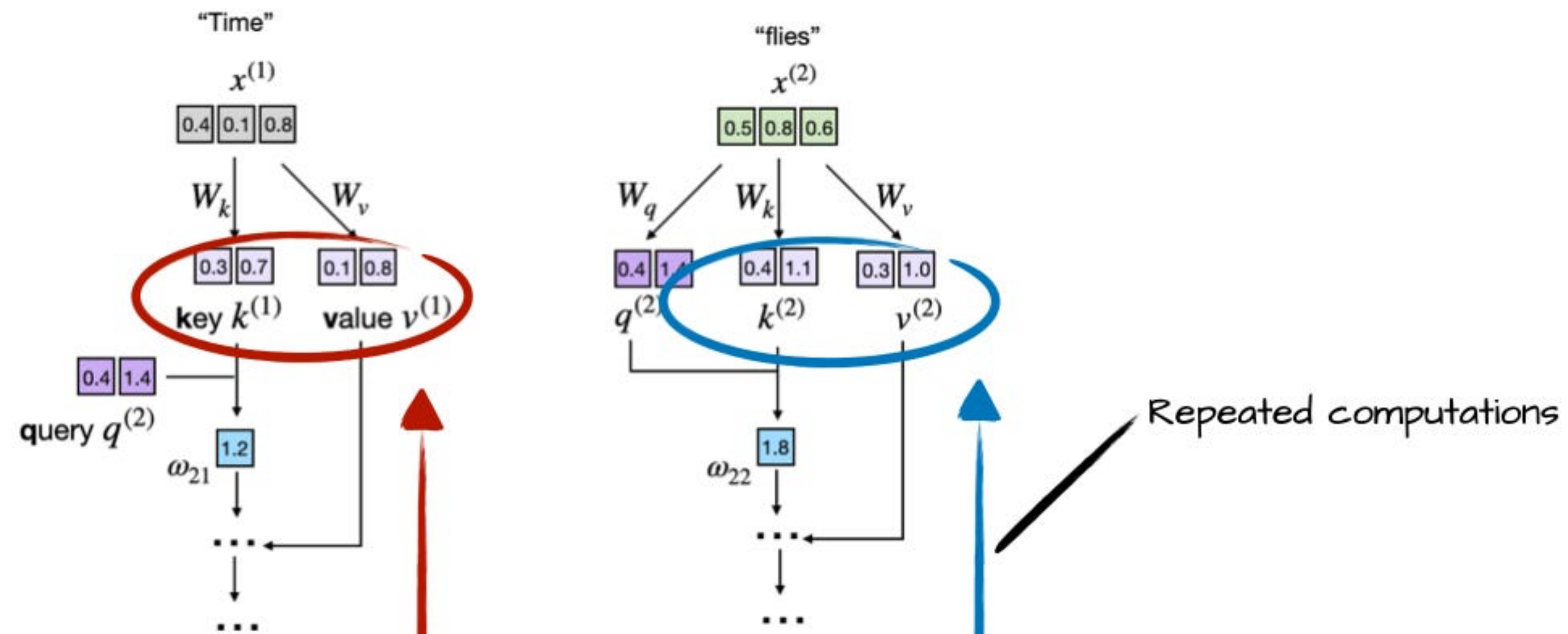
KV Cache

Motivation

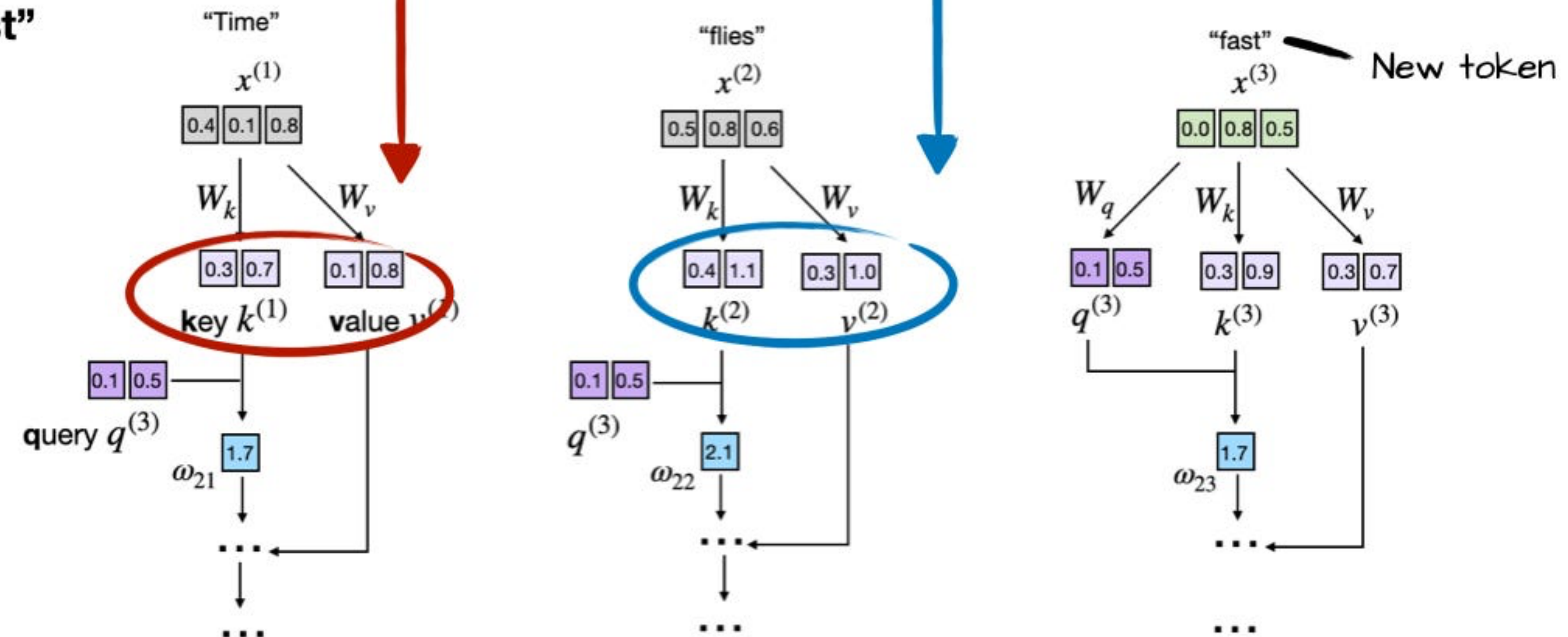
- Optimization for compute efficiency
- Store intermediate K and V computations for reuse during inference
 - Caching mechanism that stores the previously generated key and value vectors
- Avoid unnecessary re-computations



“Time flies”



“Time flies fast”



KV Cache

$$QK^{\top} = \begin{bmatrix} \mathbf{q}_0 \cdot \mathbf{k}_0 & \mathbf{q}_0 \cdot \mathbf{k}_1 & \mathbf{q}_0 \cdot \mathbf{k}_2 & \mathbf{q}_0 \cdot \mathbf{k}_3 \\ \mathbf{q}_1 \cdot \mathbf{k}_0 & \mathbf{q}_1 \cdot \mathbf{k}_1 & \mathbf{q}_1 \cdot \mathbf{k}_2 & \mathbf{q}_1 \cdot \mathbf{k}_3 \\ \mathbf{q}_2 \cdot \mathbf{k}_0 & \mathbf{q}_2 \cdot \mathbf{k}_1 & \mathbf{q}_2 \cdot \mathbf{k}_2 & \mathbf{q}_2 \cdot \mathbf{k}_3 \\ \mathbf{q}_3 \cdot \mathbf{k}_0 & \mathbf{q}_3 \cdot \mathbf{k}_1 & \mathbf{q}_3 \cdot \mathbf{k}_2 & \mathbf{q}_3 \cdot \mathbf{k}_3 \end{bmatrix}$$

$$V = \begin{bmatrix} \mathbf{v}_0 \\ \mathbf{v}_1 \\ \mathbf{v}_2 \\ \mathbf{v}_3 \end{bmatrix}$$



KV Cache

Drawbacks

- Complex to the code
 - Increased memory requirements
 - Can't be used during training
-
- Inference speed-ups are often worth the trade-offs in production



MHA

Parameter Counts ~1B Model

d	Embedding Dim	2048
n	Context Len	2048
d _h	Q K V Dim	128
h, l	Heads, Layers	16

Component	Calculation	Params (per layer) (M)	Total Params (M)	Memory (MB)
K	$d_h * d * h * l$	4	67	256
Q	$d_h * d * h * l$	4	67	256
V	$2 * d_h * d * h * l$	8	134	512
KV Cache	$n * l * 2 * d_h * h$	8	134	512

4 bytes at FP 32, 2 bytes at FP16/BF16, 1 byte at FP8/INT8 per cached element

Attention

Alternatives/Research

- Mechanism Changes
 - Longformer
 - Grouped Query Attention - LLaMA, Qwen
 - Multi Head Latent Attention - DeepSeek, Kimi
 - Linear Attention
 - Scaled Attention
 - Scaled Delta Nets
- Compute Optimization
 - Flash Attention
 - Ring Attention



Longformer Attention

Longformer

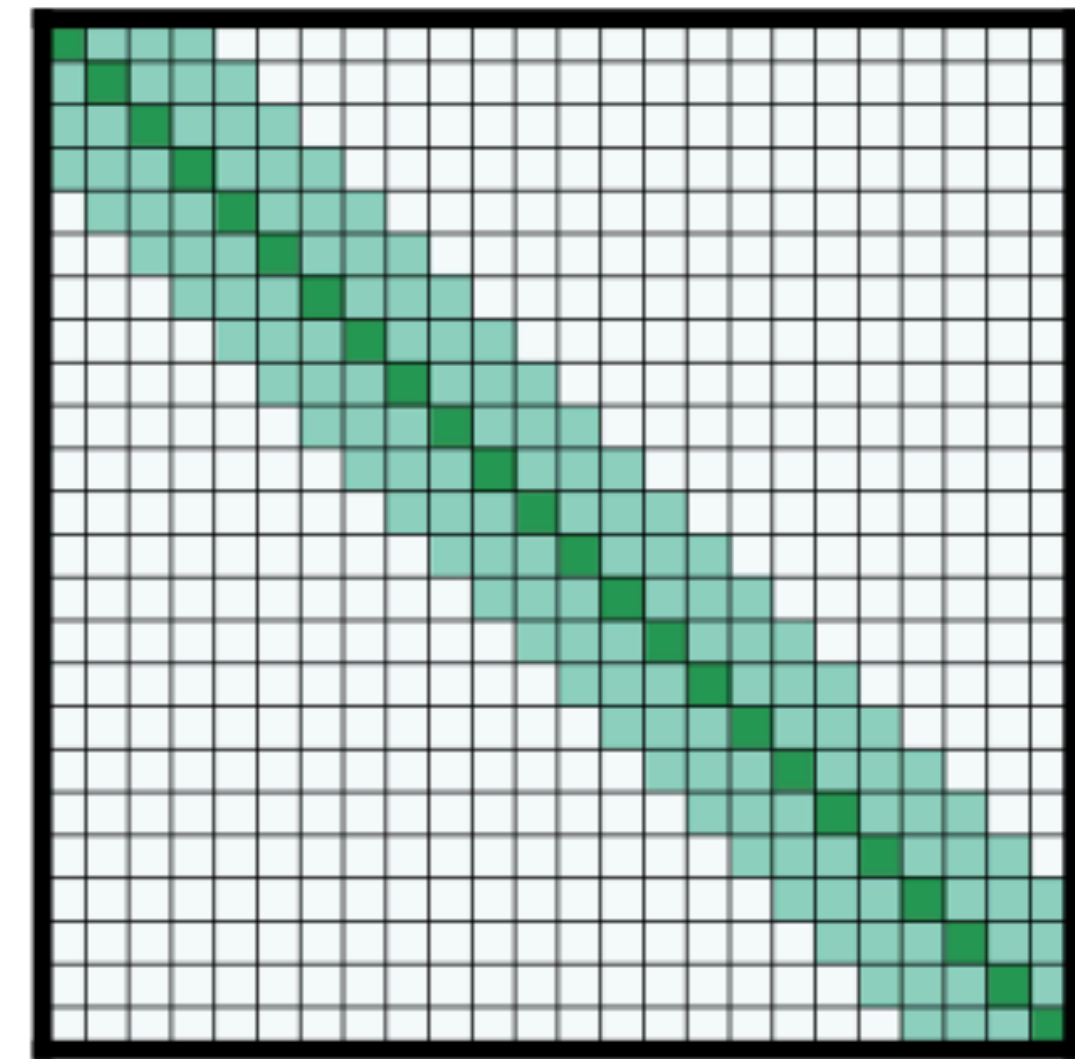
Motivation

- Solve quadratic scaling of self-attention
- Introduce attention mechanism that linearly scales with sequence length
- Useful for long documents
- Sparsify the full self-attention matrix according to an “attention pattern”
- Local windowed attention with task-specific global attention

Longformer

Attention Patterns - Sliding Window

- Fixed window size w
- Each token attends to $1/2 w$ tokens on each side
- $O(n \times w)$
- May use different values of w for each layer to balance between efficiency and model representation capacity
- Similar to CNN

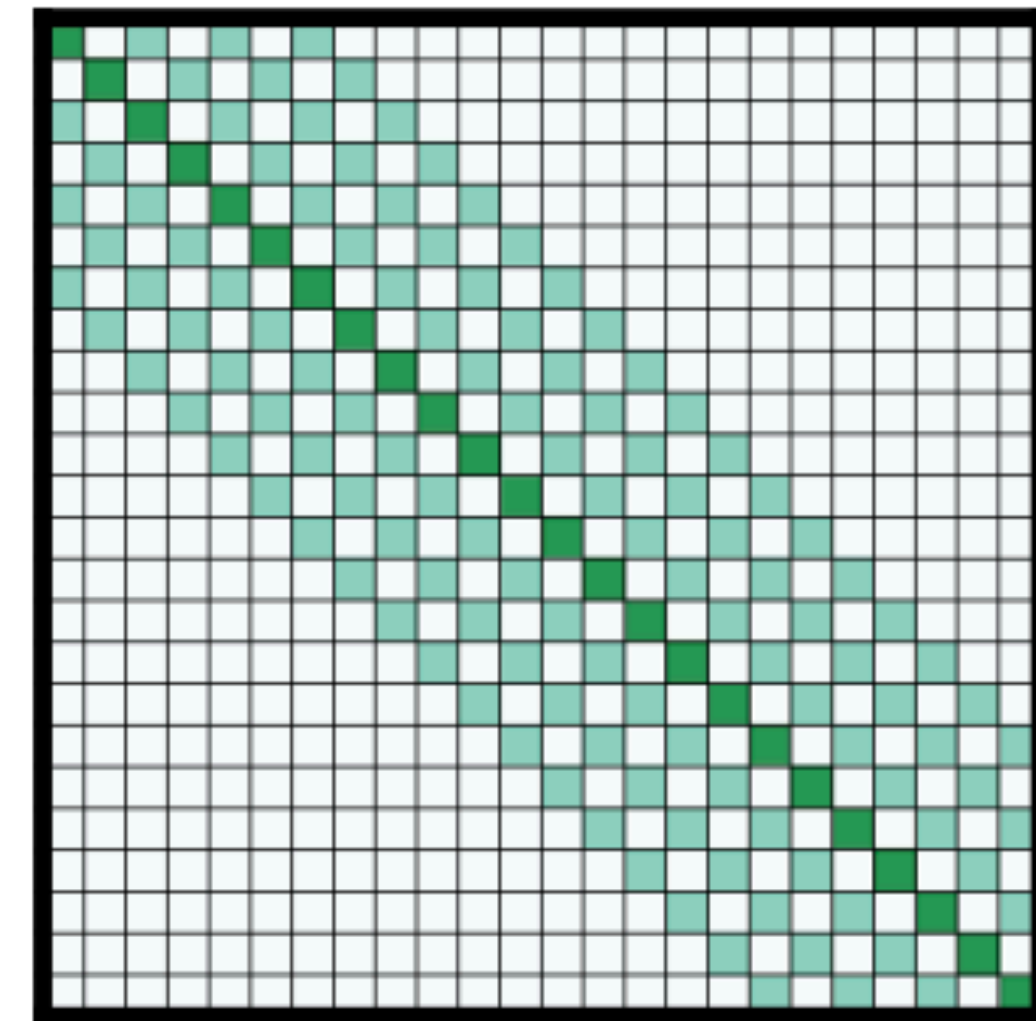


(b) Sliding window attention

Longformer

Attention Patterns - Dilated Sliding Window

- Window has gaps of size dilation d
- Different dilation configurations per head
 - without dilation to focus on local context
 - with dilation to focus on longer context
- improved performance

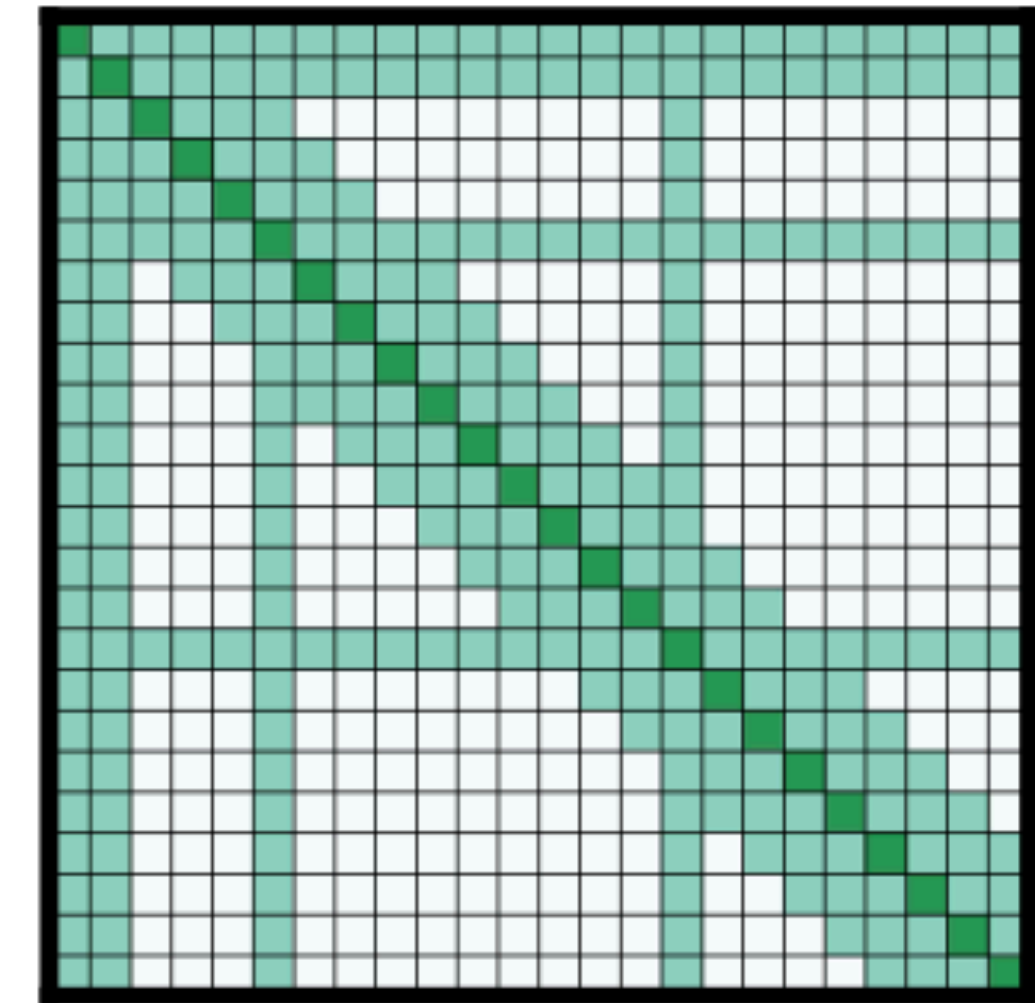


(c) Dilated sliding window

Longformer

Attention Patterns - Global Attention

- Local attention are not flexible enough to learn task-specific representations
- Global attention on few pre-selected inputs
 - attends to all tokens across the sequence
 - all tokens in the sequence attend to it
- For example
 - Classification - [CLS] token
 - QA - all question tokens



(d) Global+sliding window

Longformer

Gains

- $O(n)$ complexity on combined local and global attentions

Longformer

Drawbacks

- Potential for information loss
 - Small window sizes for the lower layers and larger sizes for higher layers
 - Top layers may learn higher-level representation of the entire sequence while lower layers capture local information
- Task specific global attention studied only for QA, Coreference Resolution, Document classification
- May not have significant improvement in tasks that need shorter local context

Multi Query Attention

Multi Query Attention

Motivation

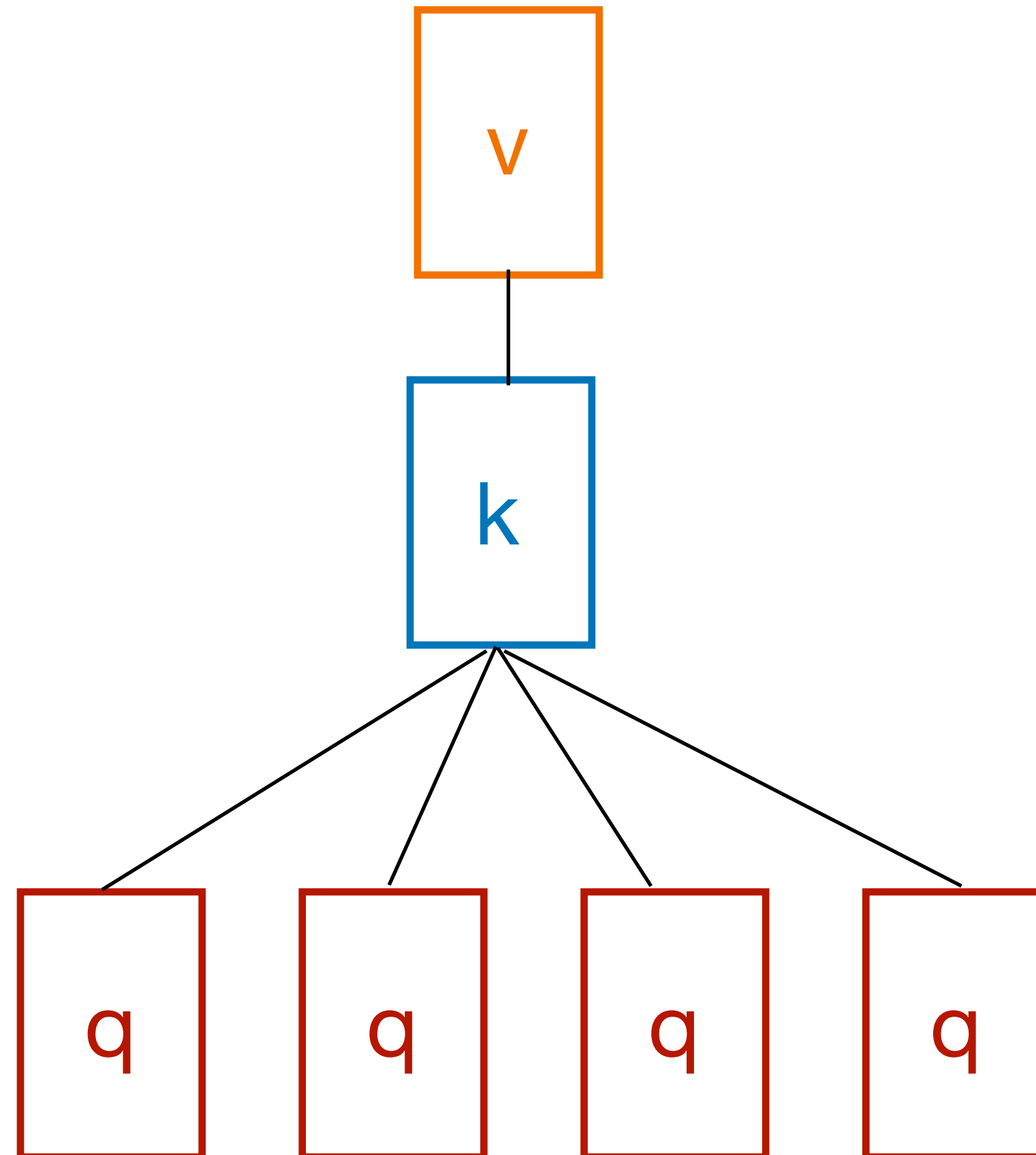
- Slow incremental inference (not parallelizable)
 - repeated loading of key and value tensors
- Reduce size of Key and Value tensors

Multi Query Attention

Approach

- Multiple Query Heads
- Single Key and Value shared across heads

Multi Query Attention



MQA

Parameter Counts ~1B Model

d	Embedding Dim	2048
n	Context Len	2048
d _h	Q K V Dim	128
h, l	Heads, Layers	16

Component	Calculation	Params (per layer) (M)	Total Params (M)	Memory (MB)
K	$d_h * d * l$	0.26	4	16
Q	$d_h * d * h * l$	4	67	256
V	$2 * d_h * d * l$	0.52	8	32
KV Cache	$n * l * 2 * d_h$	0.524	8	32

4 bytes at FP 32, 2 bytes at FP16/BF16, 1 byte at FP8/INT8 per cached element

Multi Query Attention

Gains

- Much lower memory-bandwidth requirements in the incremental setting

Multi Query Attention

Drawbacks

- Issues with using MQA for training
 - Reduced quality and capacity due to shared K, V
 - Maybe too simplified to learn and find optimal parameters
- Address only reloading key-value during inference
- Train a model just for faster inference?



Grouped Query Attention

Grouped Query Attention

Motivation

- Mitigate quality degradation of MQA
- Achieve quality close to MHA and speed to MQA

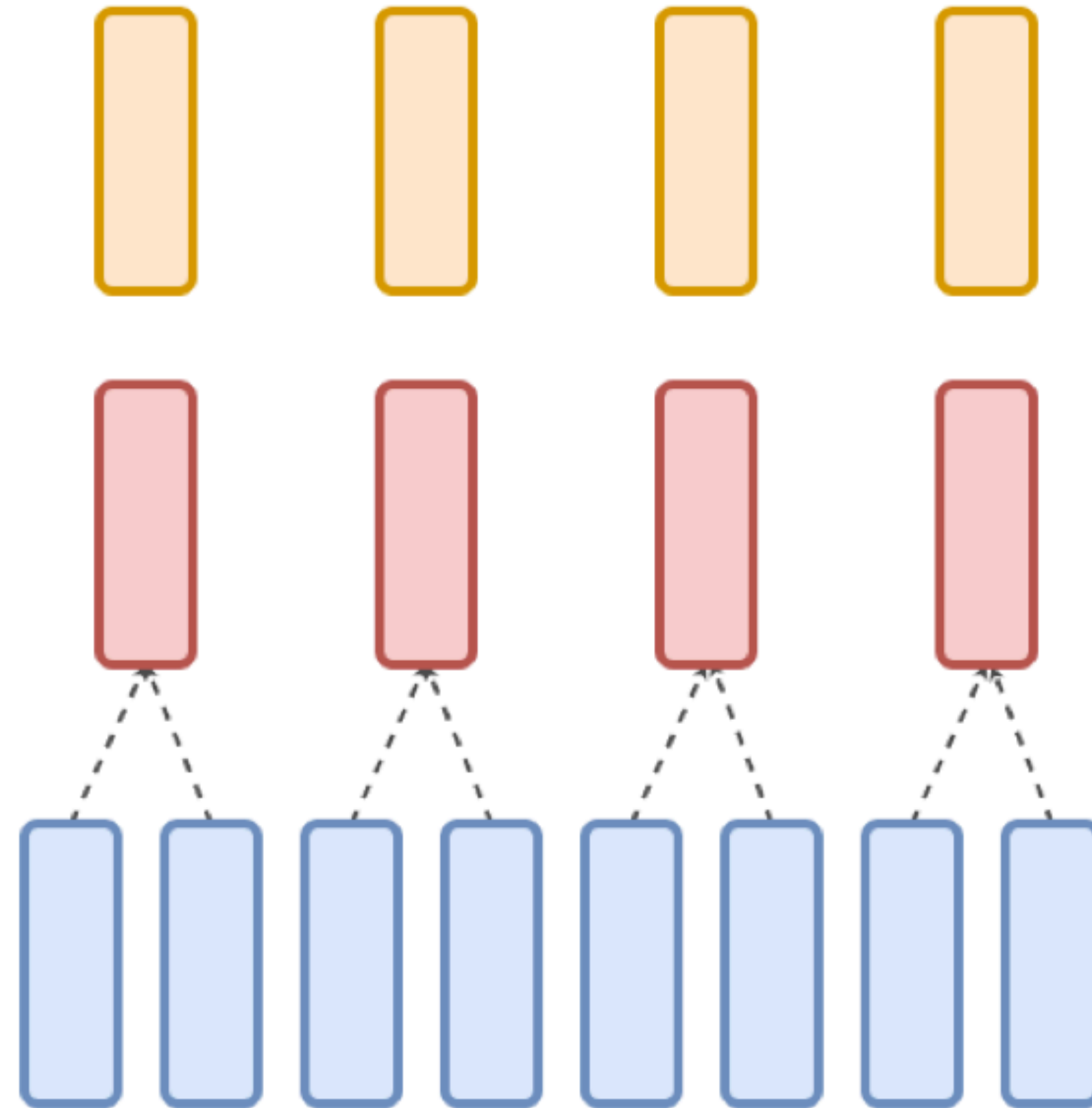
Interpolation between multi-head and multi-query attention with single key and value heads per subgroup of query heads

Grouped Query Attention

Approach

- MHA - every head has own set of keys and values
- GQA - multiple heads share same key and value projections
- Convert a multi-head checkpoint into a multi-query checkpoint
- Divide query heads into g groups, each of which shares a single key head and value head

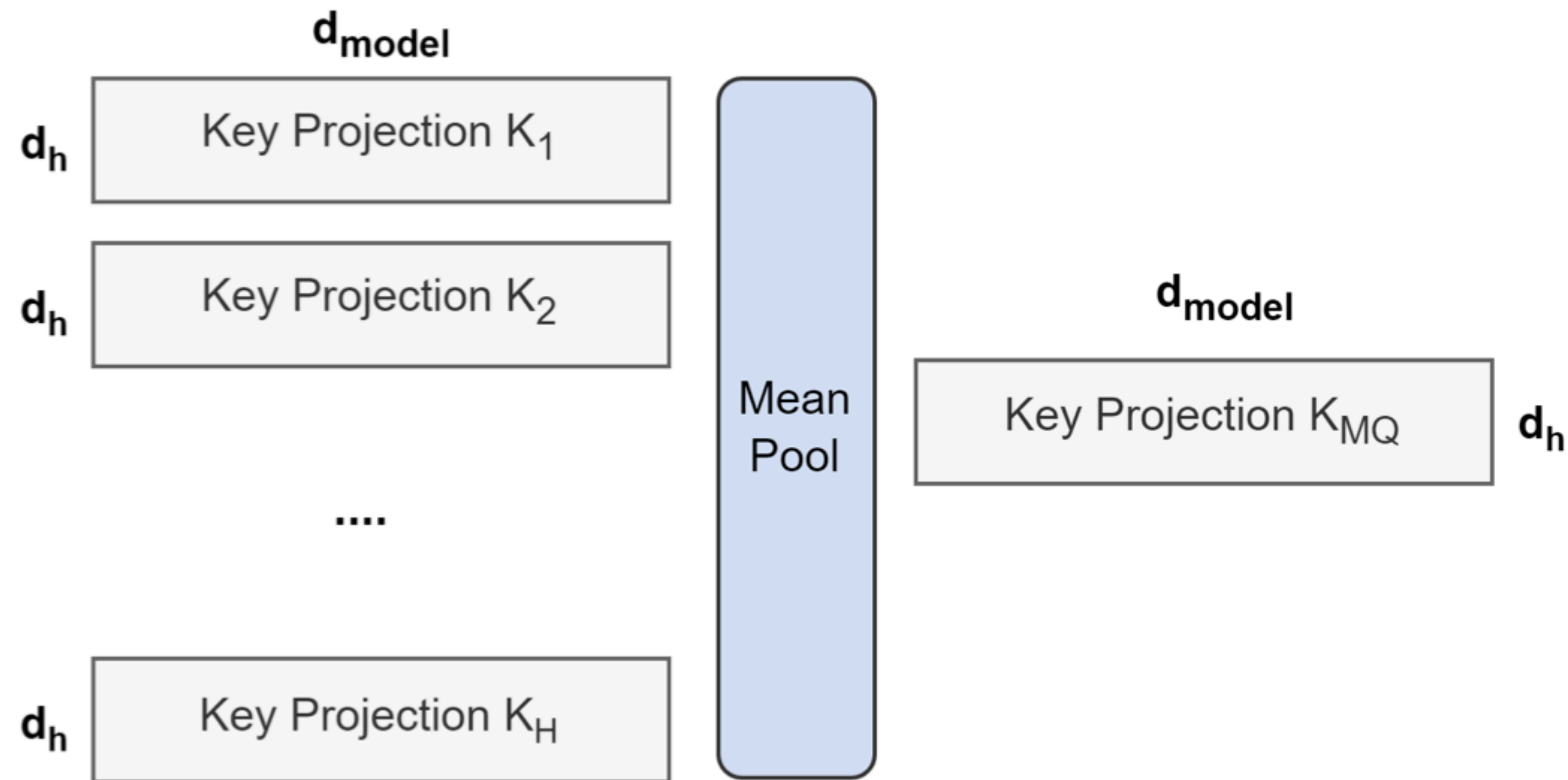
Grouped Query Attention



Grouped-query attention shares single key and value heads for each group of query heads

Grouped Query Attention

Multi head to Multi query conversion



Key and value projection matrices from all heads
are mean pooled into a single head.

GQA

Parameter Counts ~1B Model

d	Embedding Dim	2048
n	Context Len	2048
d _h	Q K V Dim	128
h, l	Heads, Layers	16
n _g	Groups (4-8)	4

Component	Calculation	Params (per layer) (M)	Total Params (M)	Memory (MB)
K	$d_h * d * n_g * l$	1	16	64
Q	$d_h * d * h * l$	4	67	256
V	$2 * d_h * d * n_g * l$	2	33	128
KV Cache	$n * l * 2 * n_g * d_h$	2	33	128

4 bytes at FP 32, 2 bytes at FP16/BF16, 1 byte at FP8/INT8 per cached element

Grouped Query Attention

Gains

- Quality close to MHA
- Comparable speed to MQA

Grouped Query Attention

Drawbacks

- More complex than MHA or MQA
 - Group management
 - K-V assignment to right group
- Find optimal G - number of groups



Multi Head Latent Attention

Multi Head Latent Attention

Motivation

- Reduce KV cache
- Mitigate performance degradation of MQA and GQA over MHA

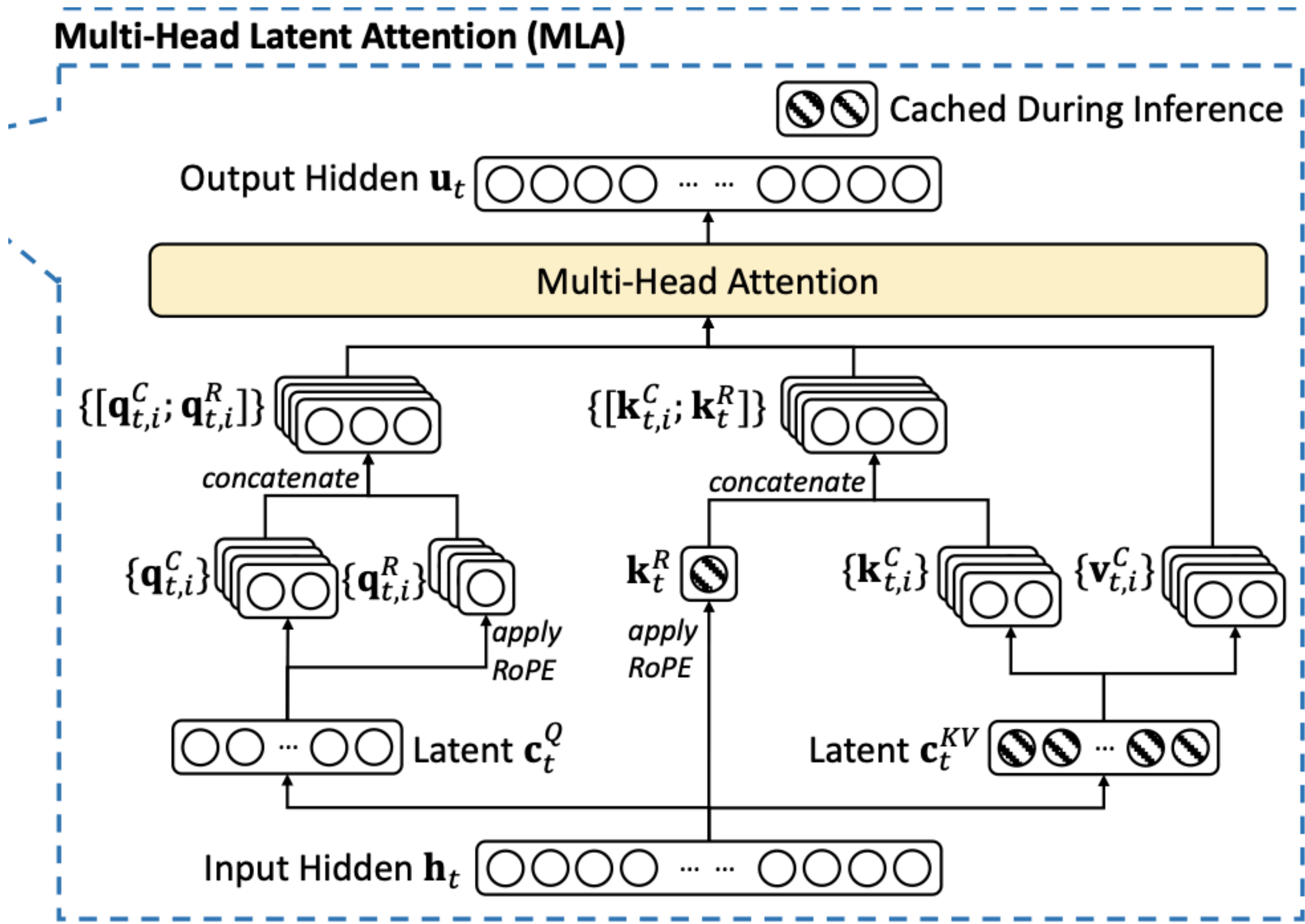
MLA vs GQA

Approach

- GQA: reduces the number of key-value heads
 - Shared Heads
- MLA: stores compressed latent representation of keys and values
 - Compressed Cache



Multi Head Latent Attention



Multi Head Latent Attention

Approach

- Low-Rank Key-Value joint compression

$$\mathbf{c}_t^{KV} = \mathbf{W}^{DKV} \mathbf{h}_t ,$$

$\mathbf{c}_t^{KV} \in \mathbb{R}^d$ - compressed latent vector for keys & values

$d_c (\ll d_h n_h)$ - KV compression dimension

$$\mathbf{k}_t^C = \mathbf{W}^{UK} \mathbf{c}_t^{KV} ,$$

$\mathbf{W}^{DKV} \in \mathbb{R}^{d_c \times d}$ - down-projection matrix

$$\mathbf{v}_t^C = \mathbf{W}^{UV} \mathbf{c}_t^{KV}$$

$\mathbf{W}^{UK}, \mathbf{W}^{UV} \in \mathbb{R}^{d_h n_h \times d_c}$ - up-projection matrices for keys & values



Multi Head Latent Attention

Approach

- During inference MLA only needs to cache $\mathbf{c}_t^{KV}$
- To reduce the memory during training, low-rank compression for the queries

$$\mathbf{c}_t^Q = \mathbf{W}^{DQ} \mathbf{h}_t,$$

$\mathbf{c}_t^Q \in \mathbb{R}^{d_c}$ - compressed latent vector for queries

$d_c (\ll d_h n_h)$ - Q compression dimension

$$\mathbf{q}_t^C = \mathbf{W}^{UQ} \mathbf{c}_t^Q$$

$\mathbf{W}^{DQ} \in \mathbb{R}^{d_c \times d}$ - down-projection matrix

$\mathbf{W}^{UQ}, \mathbf{W}^{UV} \in \mathbb{R}^{d_h n_h \times d_c}$ - up-projection matrix



Multi Head Latent Attention

Performance

Attention Mechanism	KV Cache per Token (# Element)	Capability
Multi-Head Attention (MHA)	$2n_hd_hl$	Strong
Grouped-Query Attention (GQA)	$2n_gd_hl$	Moderate
Multi-Query Attention (MQA)	$2d_hl$	Weak
MLA (Ours)	$(d_c + d_h^R)l \approx \frac{9}{2}d_hl$	Stronger

Table 1 | Comparison of the KV cache per token among different attention mechanisms. n_h denotes the number of attention heads, d_h denotes the dimension per attention head, l denotes the number of layers, n_g denotes the number of groups in GQA, and d_c and d_h^R denote the KV compression dimension and the per-head dimension of the decoupled queries and key in MLA, respectively. The amount of KV cache is measured by the number of elements, regardless of the storage precision. For DeepSeek-V2, d_c is set to $4d_h$ and d_h^R is set to $\frac{d_h}{2}$. So, its KV cache is equal to GQA with only 2.25 groups, but its performance is stronger than MHA.

MLA

Parameter Counts ~1B Model

d	Embedding Dim	2048
n	Context Len	2048
d _q	Q K V Dim	128
h, l	Heads, Layers	16
d _c	Compression Dim (d _c << hd _q)	32

Component	Calculation	Params (per layer) (M)	Total Params (M)	Memory (MB)
Down Projection	$d_c * d * l$	0.065	1	4
Up Projection	$d_c * d_q * h * l$	0.065	1	4
KV Cache	$n * l * 2 * d_c$	0.131	2	8

4 bytes at FP 32, 2 bytes at FP16/BF16, 1 byte at FP8/INT8 per cached element

Multi Head Latent Attention

Gains

- Requires a significantly smaller amount of KV cache
 - During inference, MLA only needs to cache $\mathbf{c}_t^{KV}$
 - only $d_c l$ elements, d_c - compression dimension, l - number of layers
 - W^{UK} can be absorbed into W^Q , and W^{UV} can be absorbed into W^O , we even do not need to compute keys and values out for attention
- Faster Inference
- Model Quality Improvement



Multi Head Latent Attention

Drawbacks

- Pre-training from scratch
 - No conversion possibility like MHA to MQA
- Implementation Complexity



Comparison

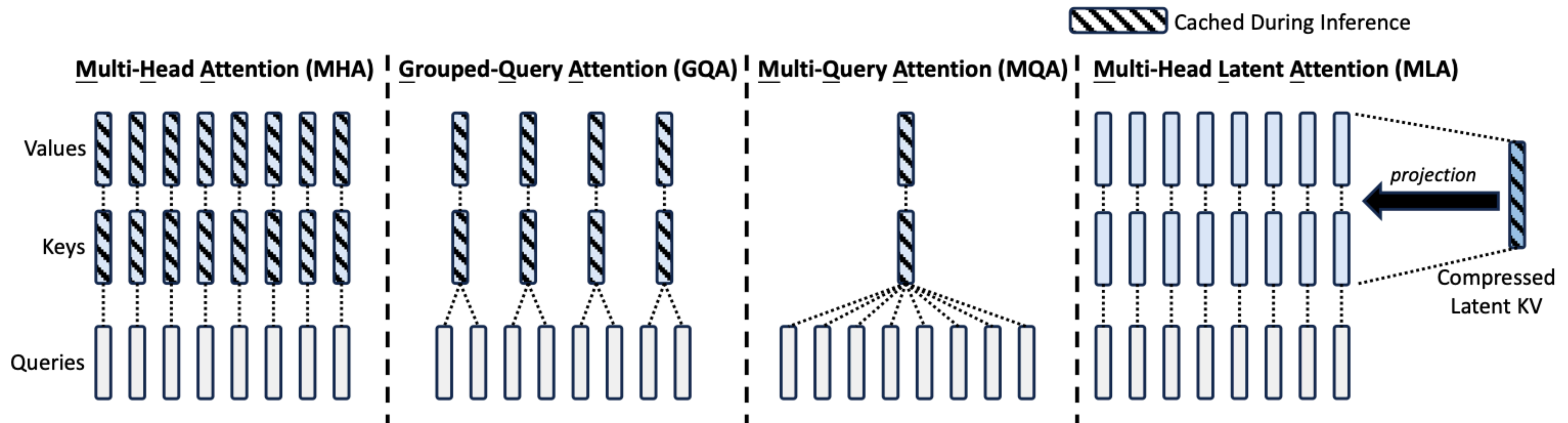
Parameter Counts ~1B Model

Attention	Calculation	Total Params (M)	Memory (MB)
MHA	$n * l * 2 * d_h * h$	134	512
MQA	$n * l * 2 * d_h$	8	32
GQA	$n * l * 2 * n_g * d_h$	33	128
MLA	$n * l * 2 * d_c$	2	8

4 bytes at FP 32, 2 bytes at FP16/BF16, 1 byte at FP8/INT8 per cached element

Summary

MHA GQA MQA MLA



Transformer Models over Years

Model	Attention
Original Base	MHA
GPT	MHA
mT5 - L	
GPT3	MHA?
LLaMA	GQA
Mistral	SWA, GQA
LLaMA 3.2	GQA
DeepSeek V3	MLA
Qwen 3	GQA
Kimi K2	MLA

Transformer Models over Years - 2026

Model	Attention
Qwen 3.5	Hybrid GatedDeltaNet
Sarvam	MLA with KV Norm
Nemotron 3 Ultra	GQA
Mistral 4	MLA
Gemma 4	5 SlidingWindow + 1 GQA
Qwen 3.6	Gated DeltaNet
DeepSeek V4	CompressedSparse + Heavily Compressed
Kimi k3	3 Kimi Delta Attention + 1 MLA